MiDAS Feature Breakdown Agents – Developer Guide

This guide explains how to use the **MiDAS Feature Breakdown Agents** in VSCode Copilot Chat to automate feature planning and story creation.

Introduction

Two agents help streamline your GitHub workflow:

1. **Create Feature Agent** – Creates structured GitHub feature issues
2. **Feature to Story Agent** – Breaks features into actionable development stories

---

Prerequisites

- The agent requires an authenticated GitHub MCP connection with permissions to create issues, add labels, and assign users.
- If no valid connection is configured, the agent will prompt to connect and issue creation will fail until a connection with appropriate scopes is provided.

---

1. Create Feature Agent

What It Does

Creates comprehensive GitHub feature issues with proper structure, acceptance criteria, and documentation links.

When to Use

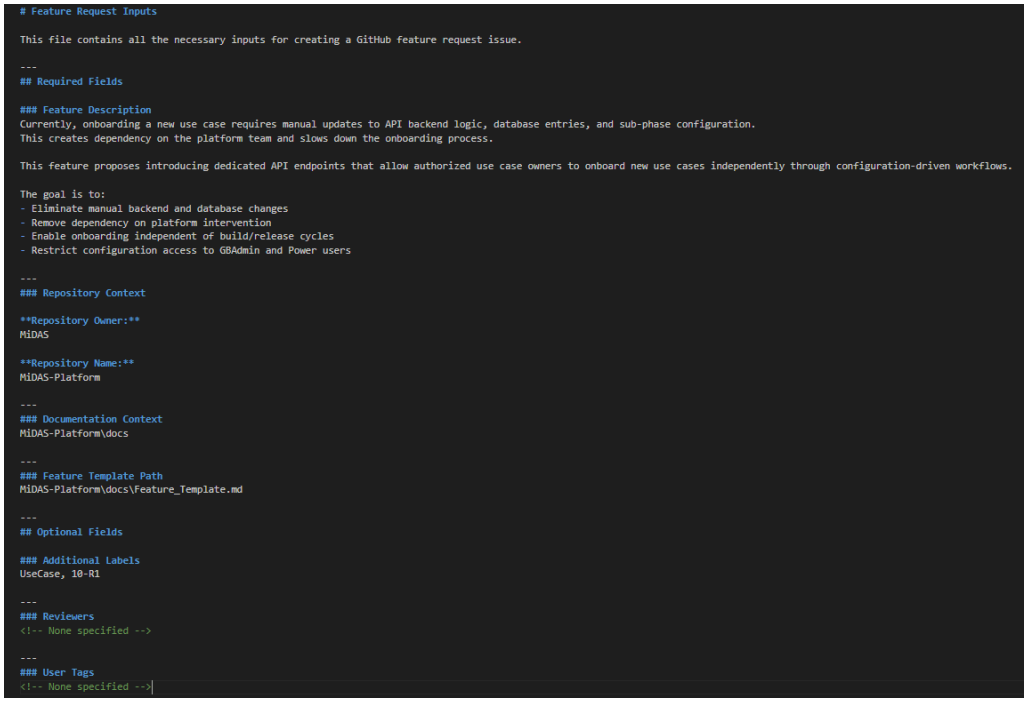
- Creating a new feature request
- Documenting feature requirements
- Ensuring consistent feature formatting

How to Use

Step 1: Prepare Input File

Create an `inputs.md` file with your feature details:

```
# Feature Request Inputs

## Required Fields

### Feature Description
<!-- Provide a concise, value-driven description of the feature to be
created -->

### Repository Context
**Repository Owner:** <!-- Default: Midas -->
**Repository Name:** <!-- Default: Midas-Platform -->

### Documentation Context
<!-- Provide one of the following: - Documentation folder path (e.g.,
/docs, /design, /architecture) - Any relevant path or link for context -
"No documentation available" if none exists -->

### Feature Template Path
<!-- Provide the path to the Feature Issue Template if one exists: -
Template path (e.g., /docs/Feature_Template.md,
.github/ISSUE_TEMPLATE/feature.md) - "No template available" if none
exists -->

## Optional Fields

### Additional Labels
<!-- List any additional labels besides "Feature" (comma-separated) -->

### Reviewers
<!-- List GitHub usernames of reviewers to assign (comma-separated) -->

### User Tags
<!-- List GitHub usernames to tag in the issue (comma-separated) -->
```

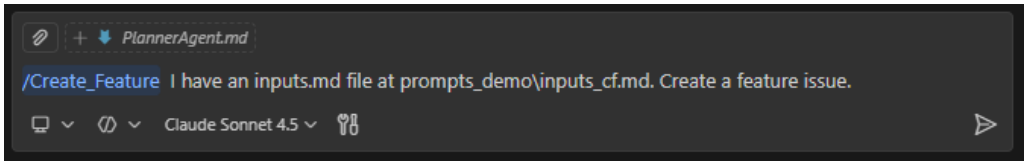


```
# Feature Request Inputs

This file contains all the necessary inputs for creating a GitHub feature request issue.

---
## Required Fields

### Feature Description
Currently, onboarding a new use case requires manual updates to API backend logic, database entries, and sub-phase configuration.
This creates dependency on the platform team and slows down the onboarding process.

This feature proposes introducing dedicated API endpoints that allow authorized use case owners to onboard new use cases independently through configuration-driven workflows.

The goal is to:
- Eliminate manual backend and database changes
- Remove dependency on platform intervention
- Enable onboarding independent of build/release cycles
- Restrict configuration access to GBAdmin and Power users

---
### Repository Context

**Repository Owner:**
MiDAS

**Repository Name:**
MiDAS-Platform

---
### Documentation Context
MiDAS-Platform\docs

---
### Feature Template Path
MiDAS-Platform\docs\Feature_Template.md

---
## Optional Fields

### Additional Labels
UseCase, 10-R1

---
### Reviewers
<!-- None specified -->

---
### User Tags
<!-- None specified -->
```

> **Note:** If no inputs.md is provided, the agent starts an interactive input flow to collect required fields. Missing required fields are prompted for and block analysis/creation until provided or explicitly skipped (skipping may reduce quality); optional fields are requested but can be skipped. The agent always summarizes collected inputs and requires an explicit "Yes" before creating any issues.

Step 2: Trigger in VSCode Copilot Chat

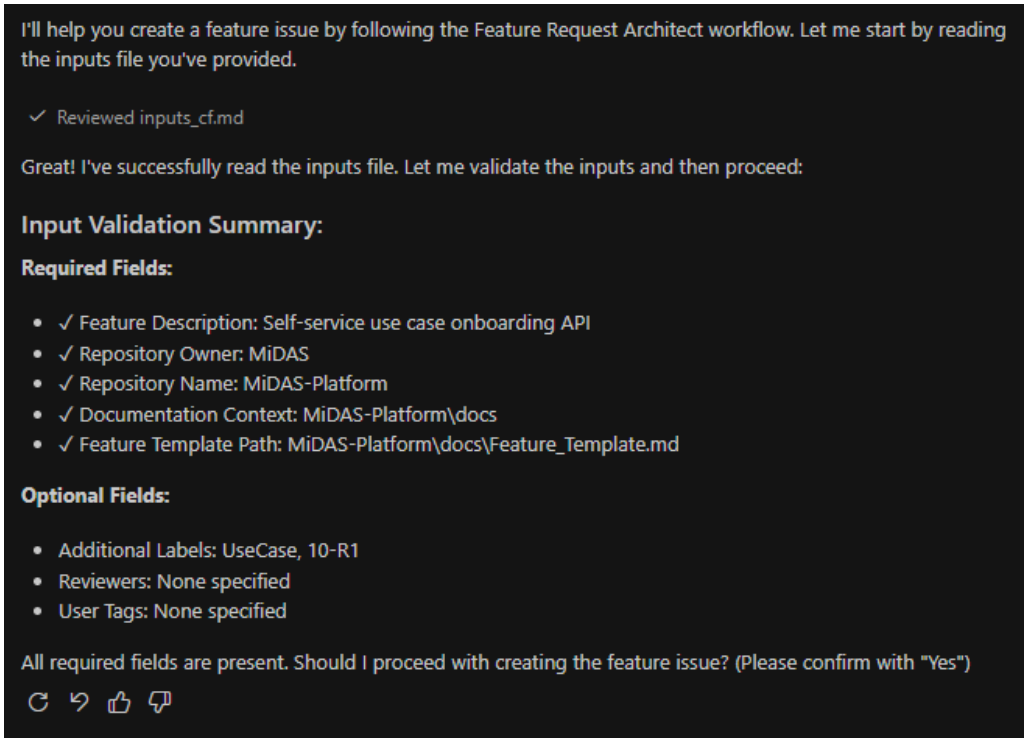
```
/Create_Feature I have an inputs.md file at /path/to/inputs.md. Create a feature issue.
```

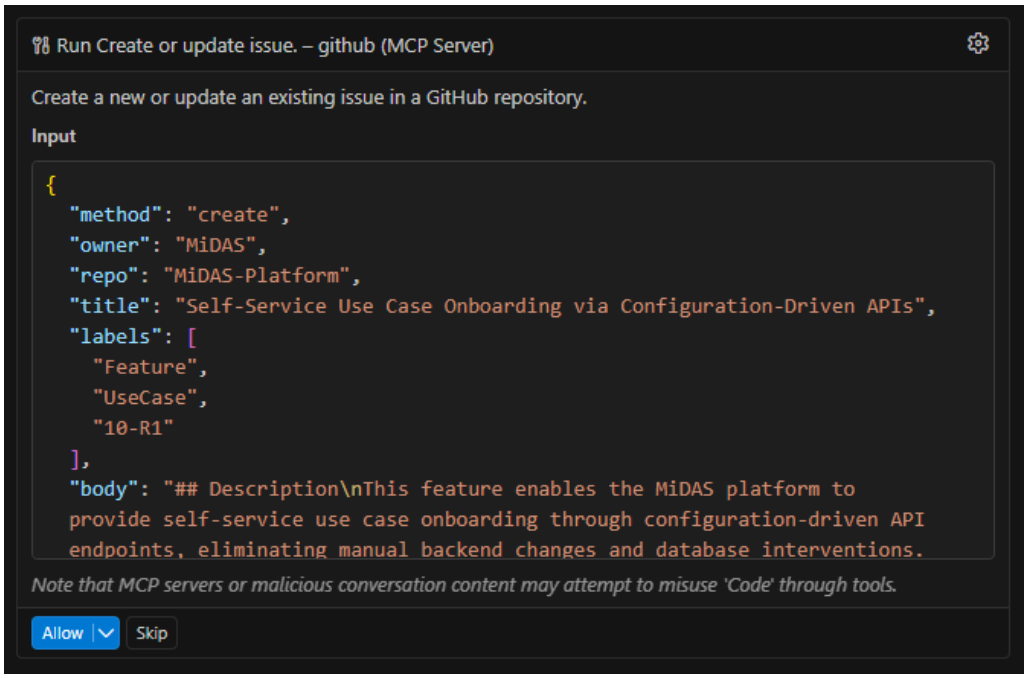


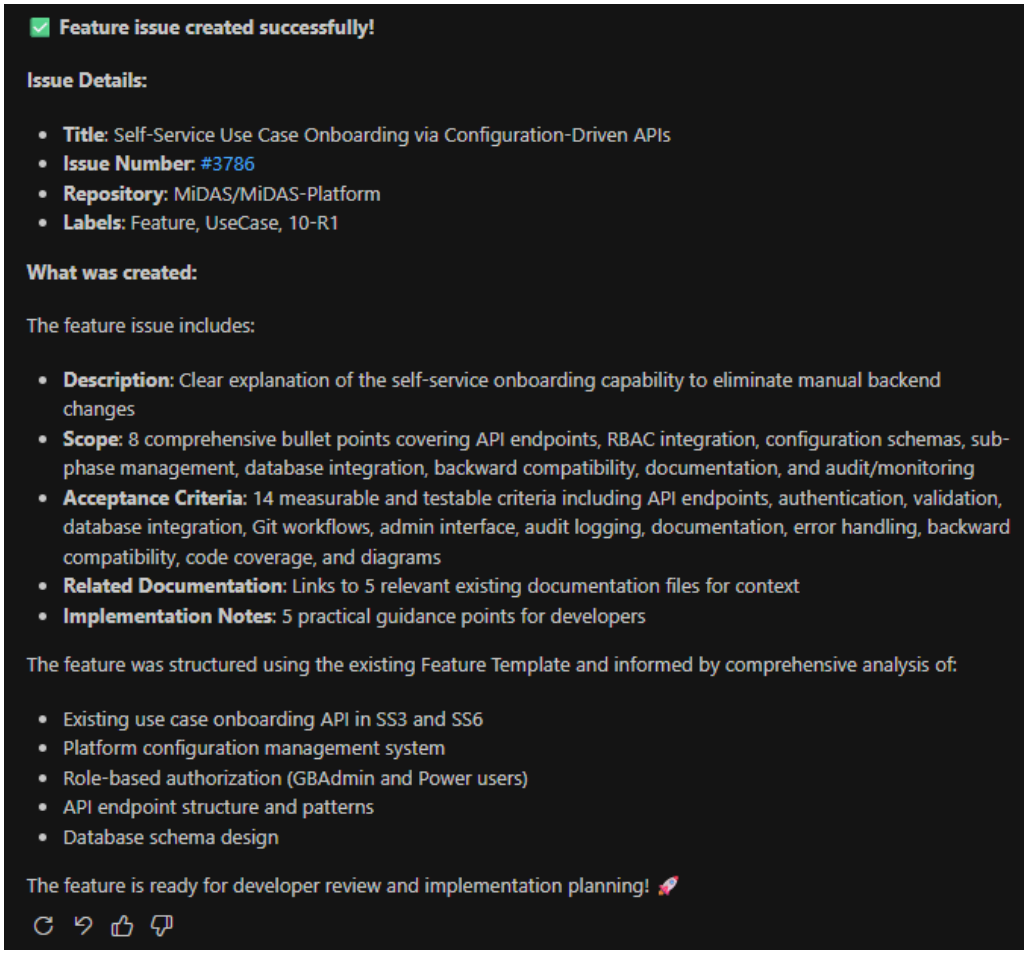
Step 3: Review and Confirm

The agent will validate inputs and ask for confirmation before creating the issue.

Ask

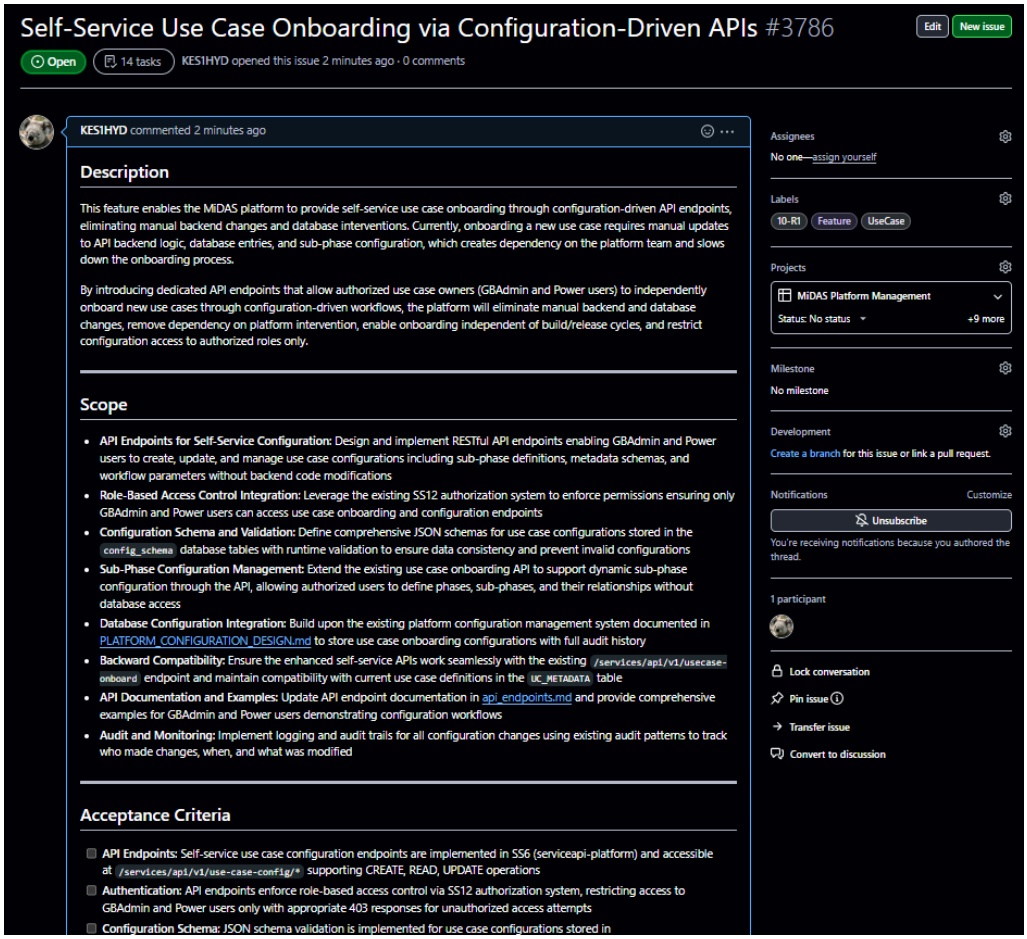

I'll help you create a feature issue by following the Feature Request Architect workflow. Let me start by reading the inputs file you've provided.
✓ Reviewed inputs_cf.md
Great! I've successfully read the inputs file. Let me validate the inputs and then proceed:
Input Validation Summary:
Required Fields:
✓ Feature Description: Self-service use case onboarding API
✓ Repository Owner: MiDAS
✓ Repository Name: MiDAS-Platform
✓ Documentation Context: MiDAS-Platform\docs
✓ Feature Template Path: MiDAS-Platform\docs\Feature_Template.md
Optional Fields:
Additional Labels: UseCase, 10-R1
Reviewers: None specified
User Tags: None specified
All required fields are present. Should I proceed with creating the feature issue? (Please confirm with "Yes")


Run Create or update issue. – github (MCP Server)
Create a new or update an existing issue in a GitHub repository.
Input
{
"method": "create",
"owner": "MiDAS",
"repo": "MiDAS-Platform",
"title": "Self-Service Use Case Onboarding via Configuration-Driven APIs",
"labels": [
"Feature",
"UseCase",
"10-R1"
],
"body": "## Description\nThis feature enables the MiDAS platform to
provide self-service use case onboarding through configuration-driven API
endpoints, eliminating manual backend changes and database interventions.
Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.
Allow
Skip



Step 4: Feature Created

A new GitHub feature issue is created with proper structure and labels.



2. Feature to Story Agent

What It Does

Breaks down feature issues into multiple development stories with clear tasks and acceptance criteria.

When to Use

- Feature needs implementation planning
- Breaking work into smaller deliverables
- Creating developer-ready tasks

How to Use

Step 1: Prepare Input File

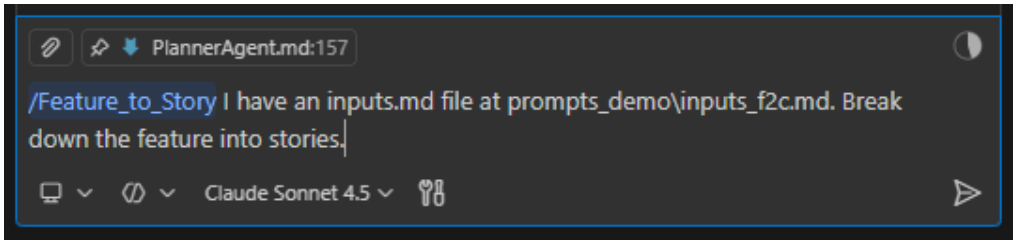
Create an `inputs.md` file with the feature reference:

```
# Story Breakdown Inputs

## Required Fields

### Feature Issue Reference
<!-- Provide the GitHub feature issue URL or issue number:
- Full URL: https://github.com/owner/repo/issues/123
- Issue number: #123 (will use default repository)
-->

### Repository Context
**Repository Owner:** <!-- Default: Midas -->
**Repository Name:** <!-- Default: Midas-Platform -->

## Optional Fields

### Impact Points
<!-- List specific areas of focus or implementation priorities (comma-
separated or line-separated):
Examples:
- authentication module
- user interface
- API endpoints
- database schema
-->

### Additional Labels
<!-- List any additional labels besides "Story" (comma-separated) -->

### Assignees
<!-- List GitHub usernames of assignees (comma-separated) -->
```


```
# Story Breakdown Inputs

This file contains all the necessary inputs for breaking down a feature into development stories.

---

## Required Fields

### Feature Issue Reference
https://github.boschdevcloud.com/MiDAS/MiDAS-Platform/issues/3786

### Repository Context

**Repository Owner:**
MiDAS

**Repository Name:**
MiDAS-Platform

---

## Optional Fields

### Impact Points
- API backend logic automation
- Database configuration automation
- Sub-phase configuration creation
- Role-based access control (GBAdmin, Power User)
- Build/release cycle independence

### Additional Labels
Story, 10-R1

### Assignees
<!-- None specified -->
```

Step 2: Trigger in VSCode Copilot Chat

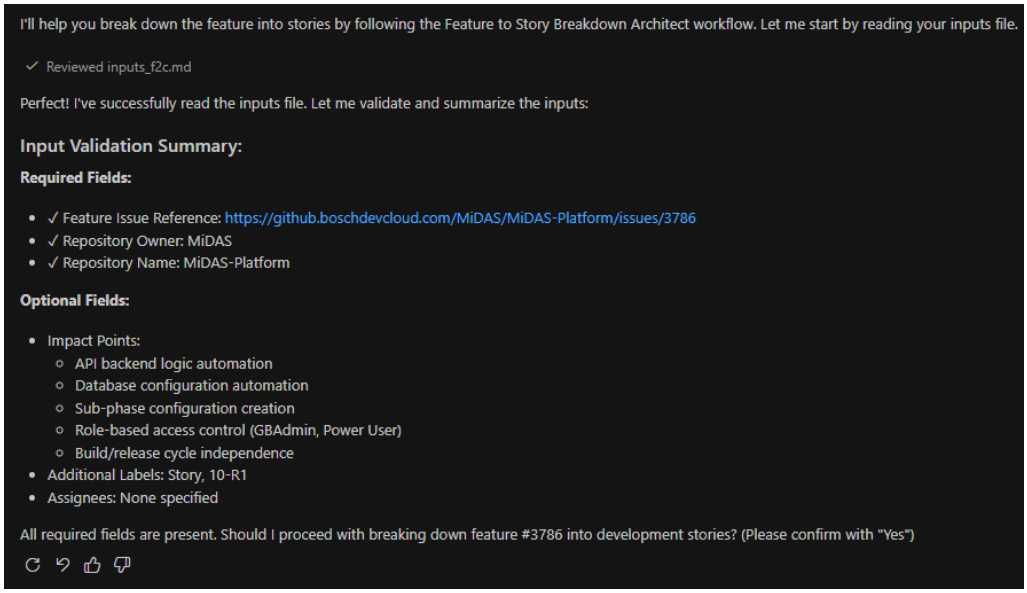
```
/Feature_to_Story I have an inputs.md file at /path/to/inputs.md. Break
down the feature into stories.
```

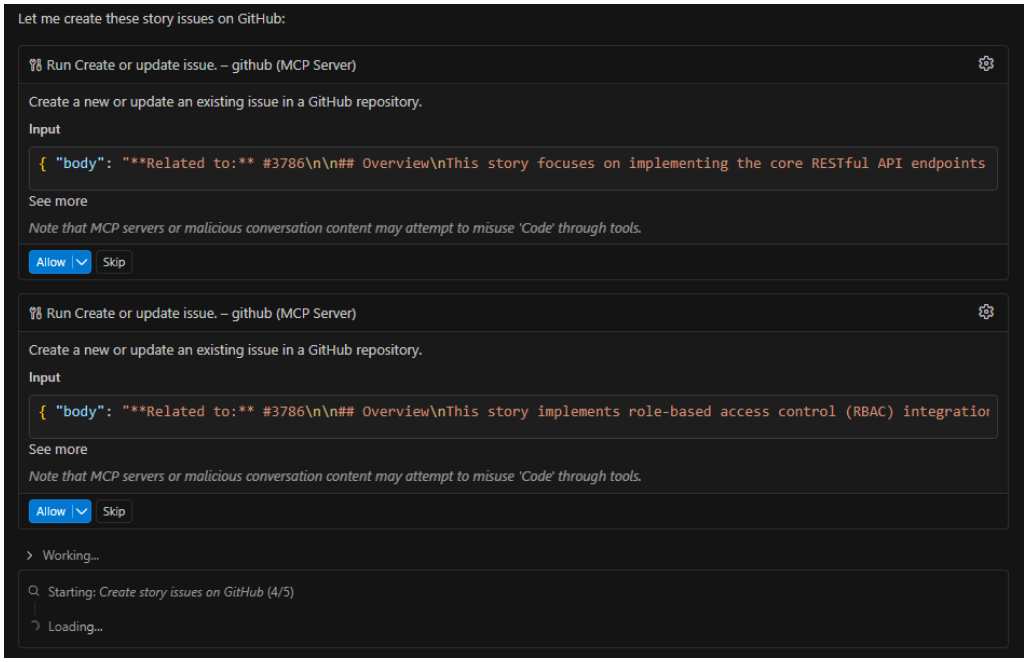


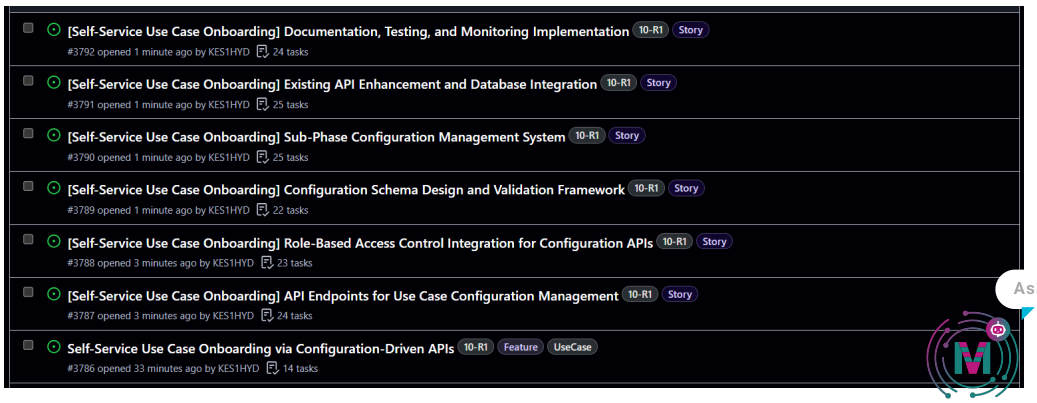
Step 3: Review and Confirm

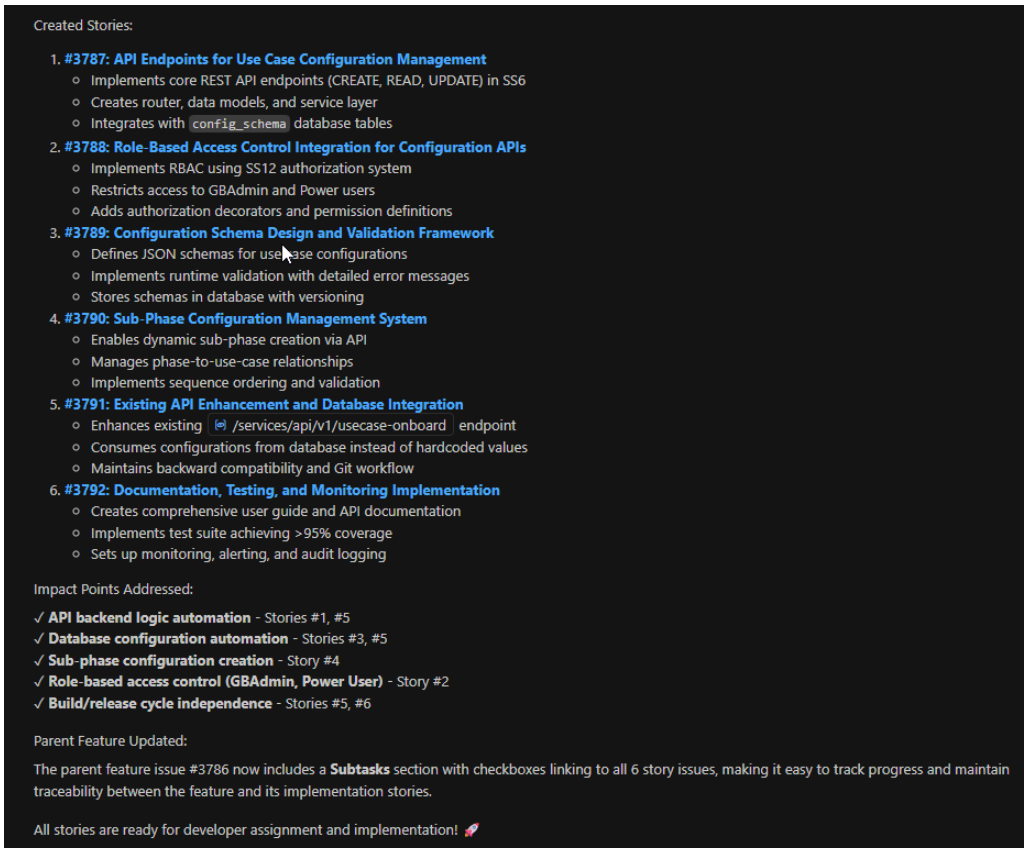
The agent analyzes the feature and asks for confirmation before creating stories.

Ask


I'll help you break down the feature into stories by following the Feature to Story Breakdown Architect workflow. Let me start by reading your inputs file.
✓ Reviewed inputs_f2c.md
Perfect! I've successfully read the inputs file. Let me validate and summarize the inputs:
Input Validation Summary:
Required Fields:
✓ Feature Issue Reference: https://github.boschdevcloud.com/MiDAS/MiDAS-Platform/issues/3786
✓ Repository Owner: MiDAS
✓ Repository Name: MiDAS-Platform
Optional Fields:
Impact Points:
API backend logic automation
Database configuration automation
Sub-phase configuration creation
Role-based access control (GBAdmin, Power User)
Build/release cycle independence
Additional Labels: Story, 10-R1
Assignees: None specified
All required fields are present. Should I proceed with breaking down feature #3786 into development stories? (Please confirm with "Yes")

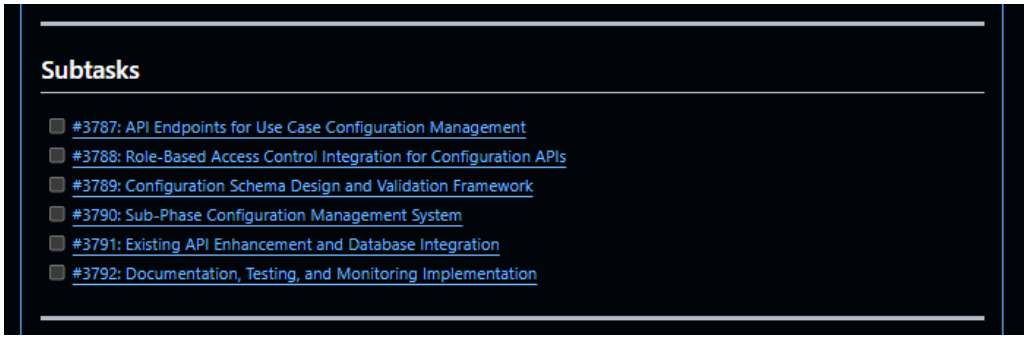

Let me create these story issues on GitHub:
Run Create or update issue. – github (MCP Server)
Create a new or update an existing issue in a GitHub repository.
Input
{ "body": "**Related to:** #3786\n\n## Overview\nThis story focuses on implementing the core RESTful API endpoints
See more
Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.
Allow
Skip
Run Create or update issue. – github (MCP Server)
Create a new or update an existing issue in a GitHub repository.
Input
{ "body": "**Related to:** #3786\n\n## Overview\nThis story implements role-based access control (RBAC) integratio
See more
Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.
Allow
Skip
Working...
Starting: Create story issues on GitHub (4/5)
Loading...

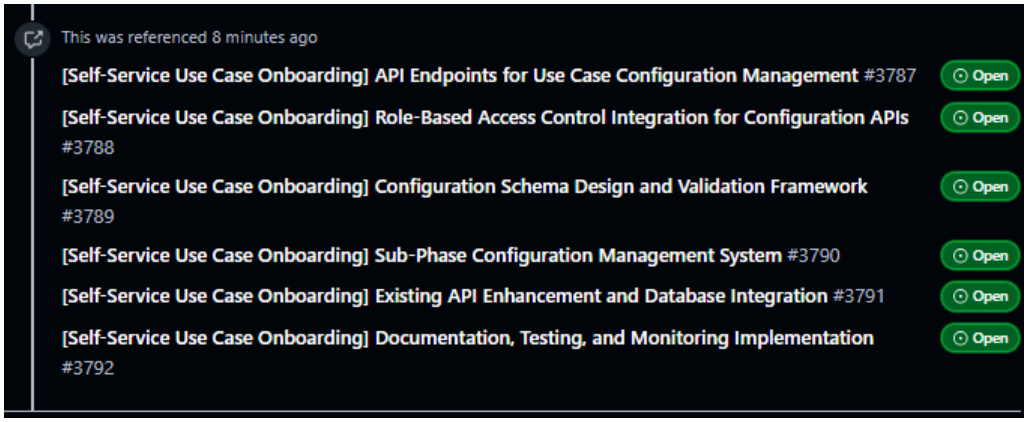

[Self-Service Use Case Onboarding] Documentation, Testing, and Monitoring Implementation 10-R1 Story
#3792 opened 1 minute ago by KES1HYD 24 tasks
[Self-Service Use Case Onboarding] Existing API Enhancement and Database Integration 10-R1 Story
#3791 opened 1 minute ago by KES1HYD 25 tasks
[Self-Service Use Case Onboarding] Sub-Phase Configuration Management System 10-R1 Story
#3790 opened 1 minute ago by KES1HYD 25 tasks
[Self-Service Use Case Onboarding] Configuration Schema Design and Validation Framework 10-R1 Story
#3789 opened 1 minute ago by KES1HYD 22 tasks
[Self-Service Use Case Onboarding] Role-Based Access Control Integration for Configuration APIs 10-R1 Story
#3788 opened 3 minutes ago by KES1HYD 23 tasks
[Self-Service Use Case Onboarding] API Endpoints for Use Case Configuration Management 10-R1 Story
#3787 opened 3 minutes ago by KES1HYD 24 tasks
Self-Service Use Case Onboarding via Configuration-Driven APIs 10-R1 Feature UseCase
#3786 opened 33 minutes ago by KES1HYD 14 tasks
Ask



Step 4: Stories Created

Multiple story issues are created and linked to the parent feature.





Quick Reference

Agent	Input	Output
Create Feature	Feature description + repo context	GitHub feature issue
Feature to Story	Feature issue URL/number	Multiple story issues

Tips

- Keep feature descriptions clear and value-focused
- Provide documentation paths for better context
- Use impact points to focus story creation
- Review generated content before confirming

Summary

Use these agents directly in VSCode Copilot Chat to automate GitHub issue creation and ensure consistent planning across your team.